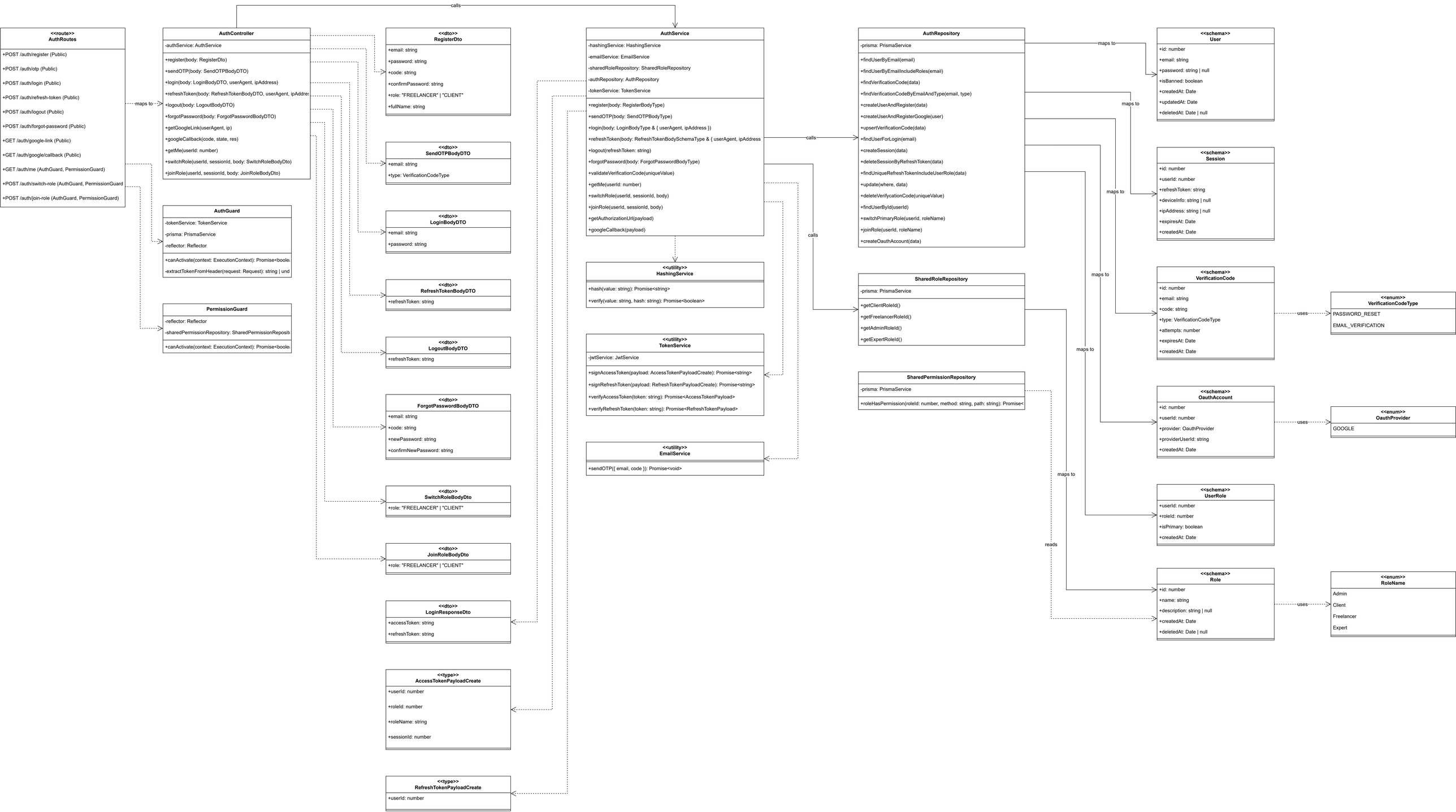

calls
<<route>>
AuthRoutes
+POST /auth/register (Public)
+POST /auth/otp (Public)
+POST /auth/login (Public)
+POST /auth/refresh-token (Public)
+POST /auth/logout (Public)
+POST /auth/forgot-password (Public)
+GET /auth/google-link (Public)
+GET /auth/google/callback (Public)
+GET /auth/me (AuthGuard, PermissionGuard)
+POST /auth/switch-role (AuthGuard, PermissionGuard
+POST /auth/join-role (AuthGuard, PermissionGuard)
maps to
AuthController
-authService: AuthService
+register(body: RegisterDto)
+sendOTP(body: SendOTPBodyDTO)
+login(body: LoginBodyDTO, userAgent, ipAddress)
+refreshToken(body: RefreshTokenBodyDTO, userAgent, ipAddre
+logout(body: LogoutBodyDTO)
+forgotPassword(body: ForgotPasswordBodyDTO)
+getGoogleLink(userAgent, ip)
+googleCallback(code, state, res)
+getMe(userId: number)
+switchRole(userId, sessionId, body: SwitchRoleBodyDto)
+joinRole(userId, sessionId, body: JoinRoleBodyDto)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boole
-extractTokenFromHeader(request: Request): string | und
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionReposit
+canActivate(context: ExecutionContext): Promise<boole
<<dto>>
RegisterDto
+email: string
+password: string
+code: string
+confirmPassword: string
+role: "FREELANCER" | "CLIENT"
+fullName: string
<<dto>>
SendOTPBodyDTO
+email: string
+type: VerificationCodeType
<<dto>>
LoginBodyDTO
+email: string
+password: string
<<dto>>
RefreshTokenBodyDTO
+refreshToken: string
<<dto>>
LogoutBodyDTO
+refreshToken: string
<<dto>>
ForgotPasswordBodyDTO
+email: string
+code: string
+newPassword: string
+confirmNewPassword: string
<<dto>>
SwitchRoleBodyDto
+role: "FREELANCER" | "CLIENT"
<<dto>>
JoinRoleBodyDto
+role: "FREELANCER" | "CLIENT"
<<dto>>
LoginResponseDto
+accessToken: string
+refreshToken: string
<<type>>
AccessTokenPayloadCreate
+userId: number
+roleId: number
+roleName: string
+sessionId: number
<<type>>
RefreshTokenPayloadCreate
+userId: number
AuthService
-hashingService: HashingService
-emailService: EmailService
-sharedRoleRepository: SharedRoleRepository
-authRepository: AuthRepository
-tokenService: TokenService
+register(body: RegisterBodyType)
+sendOTP(body: SendOTPBodyType)
+login(body: LoginBodyType & { userAgent, ipAddress })
+refreshToken(body: RefreshTokenBodySchemaType & { userAgent, ipAddress
+logout(refreshToken: string)
+forgotPassword(body: ForgotPasswordBodyType)
+validateVerificationCode(uniqueValue)
+getMe(userId: number)
+switchRole(userId, sessionId, body)
+joinRole(userId, sessionId, body)
+getAuthorizationUrl(payload)
+googleCallback(payload)
<<utility>>
HashingService
+hash(value: string): Promise<string>
+verify(value: string, hash: string): Promise<boolean>
<<utility>>
TokenService
-jwtService: JwtService
+signAccessToken(payload: AccessTokenPayloadCreate): Promise<string>
+signRefreshToken(payload: RefreshTokenPayloadCreate): Promise<string>
+verifyAccessToken(token: string): Promise<AccessTokenPayload>
+verifyRefreshToken(token: string): Promise<RefreshTokenPayload>
<<utility>>
EmailService
+sendOTP({ email, code }): Promise<void>
calls
calls
calls
AuthRepository
-prisma: PrismaService
+findUserByEmail(email)
+findUserByEmailIncludeRoles(email)
+findVerificationCode(data)
+findVerificationCodeByEmailAndType(email, type)
+createUserAndRegister(data)
+createUserAndRegisterGoogle(user)
+upsertVerificationCode(data)
+findUserForLogin(email)
+createSession(data)
+deleteSessionByRefreshToken(data)
+findUniqueRefreshTokenIncludeUserRole(data)
+update(where, data)
+deleteVerifycationCode(uniqueValue)
+findUserById(userId)
+switchPrimaryRole(userId, roleName)
+joinRole(userId, roleName)
+createOauthAccount(data)
SharedRoleRepository
-prisma: PrismaService
+getClientRoleId()
+getFreelancerRoleId()
+getAdminRoleId()
+getExpertRoleId()
SharedPermissionRepository
-prisma: PrismaService
+roleHasPermission(roleId: number, method: string, path: string): Promise<
maps to
maps to
maps to
maps to
maps to
maps to
reads
<<schema>>
User
+id: number
+email: string
+password: string | null
+isBanned: boolean
+createdAt: Date
+updatedAt: Date
+deletedAt: Date | null
<<schema>>
Session
+id: number
+userId: number
+refreshToken: string
+deviceInfo: string | null
+ipAddress: string | null
+expiresAt: Date
+createdAt: Date
<<schema>>
VerificationCode
+id: number
+email: string
+code: string
+type: VerificationCodeType
+attempts: number
+expiresAt: Date
+createdAt: Date
<<schema>>
OauthAccount
+id: number
+userId: number
+provider: OauthProvider
+providerUserId: string
+createdAt: Date
<<schema>>
UserRole
+userId: number
+roleId: number
+isPrimary: boolean
+createdAt: Date
<<schema>>
Role
+id: number
+name: string
+description: string | null
+createdAt: Date
+deletedAt: Date | null
uses
uses
uses
<<enum>>
VerificationCodeType
PASSWORD_RESET
EMAIL_VERIFICATION
<<enum>>
OauthProvider
GOOGLE
<<enum>>
RoleName
Admin
Client
Freelancer
Expert